BioSpecInfo

Documentazione dei test — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v201** · commit `a7e6610`

Generato il 2026-10-14

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Documentazione dei test — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Versione descritta	`bsi-v201`
Scopo	Descrivere come sono organizzati i test, come eseguirli, che cosa coprono e dove restano scoperti.

---

1. Tipologie presenti

Tipo	Presente	Note
End-to-end (E2E)	✅	Playwright su Chromium headless contro un server HTTP locale. È la forma prevalente.
Verifica dei dati	✅	Confronto dei dati scientifici con una fonte indipendente
Prove di stabilità	✅	Sessioni lunghe, memoria esaurita, rete degradata
Coerenza documentazione/codice	✅	Verifica che ciò che la documentazione promette esista davvero
Unit test isolati	❌	Vedi §6
Copertura di codice strumentata	✅	`audit_copertura`, con il profilatore di Chromium: nessuna build, nessun sorgente riscritto. Vedi §6

Perché E2E e non unit test

L'applicazione è composta da script classici che operano direttamente sul DOM, senza moduli né iniezione delle dipendenze: non esistono unità isolabili senza una riscrittura profonda. I difetti che si sono effettivamente verificati — un canvas sfocato, un pulsante irraggiungibile perché il contenitore è nascosto, un fornitore AI che smette di rispondere — **non sarebbero comunque emersi** da un test unitario: vivono nell'integrazione fra codice, DOM e browser.

La scelta è quindi motivata, non subita. Ne restano i limiti, dichiarati al §6.

---

2. Come eseguire i test

Requisiti

```
npm install                      # solo playwright-core
python3 -m http.server 8899 &    # RDKit WASM richiede contesto HTTP
```

> Senza server i test falliscono tutti: aprire i file con `file://` impedisce il caricamento dei moduli WebAssembly.

Tutta la batteria, con rapporto

```
node tools/genera-evidenza.js
```

Esegue ogni banco, ne trascrive l'uscita integrale e produce `docs/evidence/RAPPORTO-VERIFICA.md` con ambiente, commit e impronte SHA-256. Esce con codice `0` solo se nessun banco fallisce.

```
node tools/genera-evidenza.js --veloce   # salta i banchi con browser
```

L'opzione `--veloce` è utile per un controllo rapido ma **non è sufficiente per un'attestazione**: i banchi saltati sono proprio quelli che misurano il comportamento reale.

Un singolo banco

```
node tools/verifica-farmaci.js           # strumento a sé stante
node tools/verifica-farmaci.js --json    # uscita leggibile da programma

cd tools/banchi && node test_spettri.js  # un banco E2E
```

I banchi E2E stanno in **`tools/banchi/`**, versionati come il resto del repository; la variabile `BSI_BANCHI` permette di indicare un'altra cartella.

> **Non è sempre stato così, ed è bene dirlo.** Fino alla versione `bsi-v168` la cartella predefinita era l'area di lavoro della sessione in cui i banchi erano stati scritti: **32 dei 39 banchi non erano nel repository**. Chi clonava e lanciava il comando documentato ne trovava sette, e non riceveva alcun avviso — perché un banco assente veniva contato ma non faceva fallire la batteria, che chiudeva con «CONFORME» e zero fallimenti.
>
> Entrambe le cose sono state corrette: i banchi sono versionati, e **un banco assente rende l'esito NON CONFORME**. Un banco che non c'è non è un banco superato.

---

3. Composizione della batteria

64 banchi, raggruppati per ciò che dimostrano.

3.1 Dati scientifici

Banco	Verifica	Controlli
`audit_farmaci`	Struttura ⟷ peso, duplicati, e la **formula ricostruita dal grafo** contro quella dichiarata da ChEMBL per ogni voce con provenienza	263 voci, 36 con fonte

Banco	Verifica	Controlli
`test_spettri`	Riconoscimento dei gruppi funzionali su molecole di riferimento	36
`test_spettri_ui`	Innesto del motore nei punti dell'app che disegnano spettri	16
`test_assi`	Convenzioni degli assi negli spettri SVG	13
`test_assi_canvas`	Convenzioni degli assi negli spettri su canvas	6
`test_costanti`	Ricerca delle costanti fisiche e rifiuto delle ambiguità	45
`audit_dati`	Coerenza dei dati tabulati	29
`test_farm_ui`	Che l'Atlante Farmaci **disegni** ogni farmaco che ha in memoria, categoria per categoria, e che una categoria **mai etichettata** compaia comunque	11
`test_astro`	I dati di astrochimica: peso molecolare ⟷ formula delle 30 molecole interstellari contro i pesi atomici IUPAC, nessun corpo celeste ripetuto, ogni domanda del quiz con una risposta valida fra le sue opzioni	13
`test_datasci`	La matematica che la sezione Data Science **mostra**: R^2 su una relazione esatta, pesi standardizzati nel rapporto vero, ROC su valori calcolabili a mano, il comportamento su rumore puro, e che **nessuna sezione resti irraggiungibile**	23
`test_simmetria`	Gruppi puntuali, modi normali, regole di selezione IR/Raman	26
`test_cheminfo`	Il motore di chemioinformatica: Tanimoto e Dice su valori calcolabili a mano, impronte di Morgan e MACCS, raggruppamento di Butina, scheletri di Bemis–Murcko, PCA, regressione kernel, divisione per scheletro, **modello nullo per rimescolamento delle etichette**, validazione incrociata raggruppata, esportazioni CSV e SDF, **contrasto dei nove pannelli**, accordo fra laboratorio e motore, frammentazione, **coppie corrispondenti su effetti noti**, ricerca per sottostruttura, tabella SAR, confronto fra modelli nei due versi, arricchimento (EF e BEDROC) su casi calcolabili a mano, intervalli conformi e il loro **rifiuto sulla classificazione**	189
`test_spettrolettore`	Il lettore di spettri e i modelli che ne seguono: ASDF simbolo per simbolo, JCAMP esplicito e compresso in DIF/DUP, picchi per prominenza su spettri **costruiti** e **su rumore puro**, curva contro lista di picchi, perdite neutre sul toluene, ESOL contro quattro valori sperimentali, quattro filtri di drug-likeness separati, foresta casuale nei due versi, **una sola porta d'ingresso** (un bottone, `accept` vuoto, instradamento provato su PDF/`.docx`/`.txt`/`.jdx`/`.csv`/binario travestito/immagine) e il distintivo di versione confrontato con `BSI_APP_VERSION`. **E la relazione fra le due tabelle IR**: `SPEC_DATA.ir` serve a studiare, `BANDE_IR` ad assegnare, non sono un doppione — ma l'ombrello delle assegnazioni deve **coprire** il riferimento per ognuno dei 23 gruppi presenti in entrambe, e non essere arbitrariamente più largo (margine ricavato dai dati, non inventato)	66
`test_nmr`	La predizione NMR assegnata per atomo: due insiemi separati di molecole (taratura e **validazione**, quest'ultima mai usata per scegliere i parametri), il conteggio dei segnali su molecole di cui si sa quanti ne danno, i rifiuti su uno SMILES illeggibile e su una stringa vuota, la **cronologia** con il riconoscimento dei doppioni per chiave InChI, e la sincronia **picco ↔ struttura ↔ 3D** verificata sugli indici, non sul numero di atomi illuminati	64

Banco	Verifica	Controlli
`test_nmr2d`	Le mappe di correlazione contate sul grafo: HSQC con il segno invertito sui CH_2, COSY fra protoni non equivalenti, HMBC a due e tre legami. **Nei due versi**: l'esafluorobenzene non dà macchie HSQC, il benzene **nessuna** macchia COSY fuori diagonale, il metano **nessuna** macchia HMBC pur avendo quattro protoni	40
`test_digitalizza`	Uno spettro estratto da una **figura**, su sette immagini costruite di cui si conoscono i centri delle gaussiane al cm^{-1}: figura nuda, con cornice ed etichette, con griglia grigia, con griglia nera, con cornice **e** griglia, tratteggiata, in %T. Scarto **medio 2,2 cm^{-1}**, **peggiore 7,3** — una colonna vale ~4 cm^{-1}, che è il limite della figura e non del metodo. **Nei due versi**: da un foglio bianco e dall'icona dell'applicazione non esce **nessuna** curva; il ripiego grezzo sulla stessa figura con la cornice trova **un** picco invece di tre; su asse logaritmico il punto di mezzo è la media geometrica e su asse lineare quella aritmetica. Conversione %T→A sui valori esatti (50 %T → A = 0,301) e **giro completo** del JCAMP esportato, riletto dal lettore stesso	45
`audit_statico`	Le classi di bug che questo progetto ha già avuto, cercate in **tutti** i file invece che dove si sono manifestate: `calc()` senza spazi (il browser scarta la regola), segreti nei log, chiavi API nel codice, password in chiaro, `target="_blank"` senza `noopener`, id duplicati nel markup, script sbilanciati. Il rapporto **segnala e non blocca**, perché quelle classi hanno falsi positivi. Tre misure invece **bloccano**, e hanno un riferimento scritto: blocchi di codice duplicati verbatim (**0**), funzioni globali ridefinite (**11**, lo stile «patch additivo» dichiarato del file), di cui con corpi identici (**0**). Se crescono, la batteria fallisce	3 cancelli
`test_articoli`	Che nessuna funzione abbia due porte d'ingresso, e che gli articoli non siano scritti a mano. Dieci schede dentro un solo Centro spettri, con ogni identificatore presente nel documento **una volta sola**; nessun id ripetuto, nessun pulsante doppio, nessuna sezione raggiungibile da due strade; Spectra fuori dal menù e con il suo pulsante dedicato. Poi gli articoli: i link provati come **funzioni pure** dell'identificatore, le spiegazioni dei Nobel abbinate all'argomento della motivazione e **nessuna** inventata quando non si riconosce, e la prova decisiva — una risposta **iniettata** con un nome inesistente, che deve comparire sullo schermo. **Tre guasti distinti**: richiesta fallita, risposta vuota, forma non riconosciuta	58
`test_geom3d`	La geometria 3D costruita dal grafo, misurata contro i valori di letteratura: legami, angoli di valenza, linearità degli alchini. **Nei due versi**: il benzene esce piano e il cicloesano **non** piano. E la prova che rende lecito il collegamento picco↔atomo: ogni atomo pesante ha **lo stesso indice e lo stesso elemento** del predittore	29
`test_documento`	L'apertura di qualunque documento e lo svolgimento del quesito che contiene: PDF, Word e testo aperti davvero e il contenuto verificato; i dati riconosciuti **con la loro provenienza**. **Nei due versi**: un'immagine dichiara che non c'è riconoscimento ottico e non inventa uno svolgimento, un formato sconosciuto viene rifiutato invece che indovinato, $CDCl_3$ non diventa la formula del composto, senza etichetta MS non ci sono masse	47

Un banco che verifica un modello deve verificarlo anche quando il modello deve fallire. `test_cheminfo` prova il modello nullo nei due versi: su un segnale apprendibile l'R^2 vero (0,394) batte tutti e otto i sosia con etichette rimescolate, con un margine di 0,287; sulle **stesse molecole con etichette casuali** l'R^2 (−0,199) **non** li batte. Un controllo che verifica solo il primo caso passerebbe anche se la guardia fosse cablata su «vero».

Un «contrasto 0» che non copriva la superficie nuova. `verifica-accessibilita` misura ciò che è visibile, e i sei pannelli della sezione Chemioinformatica restano nascosti finché l'analisi non è stata eseguita: non li aveva mai visti. Ora `test_cheminfo` esegue l'analisi, apre i pannelli a turno e misura il contrasto su **460 elementi di testo**, con le formule WCAG riscritte dentro il banco — chiedere al codice sotto esame quanto vale il proprio contrasto non è misurare. Il banco pretende inoltre di averne misurati più di 250: se i pannelli non si aprissero, lo zero sarebbe vuoto e il controllo fallirebbe.

Tre picchi trovati non dimostrano nulla da soli. Un rilevatore troppo sensibile ne trova trenta dove ce ne sono tre; uno troppo rigido non ne trova nessuno. In nessuno dei due casi compare un errore. `test_spettrolettore` parte da tre gaussiane **costruite** a 1715, 2950 e 3400 cm^{-1} sotto rumore e pretende che il lettore ne trovi tre — 1716, 2952, 3402 — e poi, sullo **stesso rumore senza gaussiane**, che ne trovi **zero**. Senza il secondo controllo, il primo passerebbe anche con un rilevatore che segna ogni oscillazione. Lo stesso vale per la foresta casuale: R^2 fuori sacco 0,90 sulla relazione vera e **−0,16 su puro rumore**.

Un predittore non si rompe mai: produce sempre dei numeri. Se sono sbagliati di venti ppm, lo spettro si disegna lo stesso e sembra uno spettro. L'unico modo di sapere se serve a qualcosa è confrontarlo con valori misurati da altri — e c'è una trappola che rende quel confronto inutile: tarare i parametri sulle stesse molecole su cui poi si misura l'errore. Il numero che esce dice allora quanto bene lo schema ricorda i propri esempi. `test_nmr` tiene due insiemi separati: **taratura** (9 molecole, scarto 0,56 ppm) e **validazione** (22 molecole mai usate per quello, **0,92 ppm**). Il valore dichiarato nel pannello è il secondo. E il banco pretende che la validazione resti **peggiore** della taratura: se un giorno fossero uguali, vorrebbe dire che qualcuno ha spostato una molecola da un insieme all'altro, e il numero non direbbe più niente.

Il difetto era nel mio contatore, non nei dati. La prima esecuzione di `verifica_farmaci_v187` ha dato **0 formule su 36**: ogni molecola risultava `C<n>`, tutti carboni e nessun idrogeno. Il formato `get_json` di RDKit è **commonchem**, dove un atomo porta soltanto i campi che **differiscono** dai valori predefiniti — `z` vale 6 e `impHs` vale 0 — così che un atomo scritto `{}` è un carbonio. Leggevo `a.element`, che in quel formato non esiste. Trentasei dati che sembrano sbagliati tutti insieme sono quasi sempre un contatore sbagliato.

3.2 Agente AI

Banco	Condizione simulata
`test_404`	Il fornitore ha ritirato il modello
`test_503`	Sovraccarico temporaneo
`test_ko` · `browser_ko`	Fornitore irraggiungibile dal browser
`test_firma`	Firme di ragionamento opache da restituire
`test_tetto`	Tetto di token appreso dall'errore
`test_attesa` · `test_attesalunga`	Timeout e attese imposte
`test_doppioinvio`	Due invii sovrapposti
`browser_prova`	Misura di raggiungibilità senza chiave
`test_nucleo` · `browser_prov`	Ciclo agentico e selezione del fornitore
`caccia_ai`	Risposte malformate, stream troncato, 502 con HTML, Stop durante la risposta, pannello chiuso a metà, testo ostile

3.3 Stabilità

Banco	Cosa mette alla prova
`audit_stabilita`	95 sezioni aperte 5 volte, memoria piena, dati corrotti, raffiche di clic
`audit_promesse`	Promesse rifiutate e non gestite, con rete sana e con rete morta
`audit_quota`	`localStorage.setItem` forzato a fallire su 10 pagine
`test_sw`	Offline, rete degradata, aggiornamento durante il lavoro
`test_filemanager`	Archivio non disponibile, spazio esaurito, scritture fallite
`test_visore3d`	Consumo di contesti WebGL sfogliando molecole

3.4 Interfaccia

`browser_reset` · `browser_proxy` · `browser_proxyui` · `browser_rdkit` · `browser_lab` · `browser_frontiera` · `test_aggiorna` · `test_guidaproxy` · `test_fluidita` · `test_lingue` · `test_mol3d` · `test_spettrolettore`

`test_fluidita` — 15 controlli. Misura quanto la pagina resta *bloccata* a ogni cambio di sezione, su tutte le 91. Ha trovato tre difetti che nessun altro banco vedeva: il «Viewer 3D PRO» che fermava la pagina **797 ms** al primo click — il profilatore li attribuisce al primo `render()` di 3Dmol, che compila gli shader del contesto WebGL; le **296 figure** di «Sintesi» (5 351 nodi di SVG) disegnate tutte prima di mostrare qualunque cosa, mentre all'apertura se ne vedono tre; e **296 timer a 40 ms di distanza** programmati all'apertura della stessa sezione — undici secondi e mezzo di risvegli che chiedevano `getTotalLength()` su ogni tratto di ogni figura, comprese quelle fuori dallo schermo.

Dopo le correzioni: cambio mediano **16 ms**, peggiore **87 ms**, **nessuna sezione oltre i 100 ms** (erano due, la peggiore a 1 166 ms).

Alla versione `bsi-v188` lo stesso banco ha intercettato due cose. La prima: con 263 farmaci invece di 233, «Farmacologia» bloccava la pagina **159 ms** e le sezioni oltre i 100 ms tornavano a essere **tre** — la batteria era passata per un soffio la volta prima, e un limite superato a intermittenza non protegge niente. Le carte ora si disegnano a fette da 8 ms: **55 ms**, zero sezioni oltre i 100.

La seconda riguarda il banco stesso. La soglia sul visore 3D era **80 ms assoluti**; con lo stesso codice pubblicato, su container diversi, lo stesso click ha misurato 14, 22, 98, 105 e 192 ms, perché WebGL qui è **SwiftShader** e il costo è la compilazione degli shader. Una soglia così misura la macchina: fallisce a codice identico e passerebbe a codice peggiorato. Anche un **rapporto** su una calibrazione nella stessa esecuzione non regge — quando il banco arriva al visore, WebGL è già caldo (2,9 ms) e il visore paga il freddo: il rapporto oscillava fra 14× e 36×. L'affermazione è diventata **strutturale**, che è anche il difetto originario: subito dopo il ritorno del gestore del click il contesto WebGL **non deve esistere**, poco dopo **deve**. Misurato 0 e 1; e la misura non è vacua, perché al secondo click — quando la tela c'è già — restituisce 1.

`test_lingue` — 64 controlli. Sorveglia due funzioni nuove e due modi diversi di mentire senza accorgersene.

La lingua. Un interruttore che «traduce l'applicazione» è facile da scrivere e difficile da mantenere onesto: basta che qualcuno cambi un'etichetta italiana e la sua traduzione non si trovi più. Il difetto non si vede — l'etichetta resta in italiano in mezzo all'inglese — a meno che qualcuno conti. Il banco conta: **160 elementi di scheletro, 160 tradotti, zero rimasti**. Pretende anche le due prove contrarie: che passando all'inglese il testo **cambi davvero** (un dizionario vuoto passerebbe «tutto tradotto» senza muovere una lettera) e che tornando all'italiano il testo sia **identico carattere per carattere**.

I linguaggi delle molecole. Le conversioni si verificano contro fatti esterni — sette chiavi InChI di letteratura **scritte dentro il banco** — e si pretende il giro completo SMILES → molfile → SMILES. Le formule si contano a mano.

Il nominatore IUPAC. 26 nomi scritti a mano in due lingue, e sette molecole che **devono essere rifiutate**: un nominatore che non rifiuta mai è un nominatore che inventa.

Provato nei due versi: rimettendo il `trim()` che rompe i molfile e togliendo una traduzione, il banco fallisce tre controlli su venti.

Le porte. Dalla v184 il banco verifica anche che le due funzioni siano **raggiungibili**: pulsante di navigazione, sezione che si disegna davvero, e — controllo nato da un difetto reale — che nessuna delle due mostri il cartello «🚧 Sezione in costruzione», che il ripiego per le sezioni vuote stampa 800 ms dopo un click in qualunque sezione lasciata vuota. Verifica inoltre che la voce del menu ✨ **porti** alla sezione e non apra una finestra sovrapposta, e che non esista alcun identificativo duplicato con entrambe le sezioni disegnate.

Le sei lingue. Per ognuna si pretendono due cose insieme: copertura piena dello scheletro (**162 su 162**) e che il testo **cambi davvero** — un dizionario vuoto passerebbe «tutto tradotto» senza muovere una lettera.

`test_mol3d` **— 46 controlli.** Sorveglia l'animazione di formazione della molecola e la misura degli angoli di legame, due aggiunte che hanno due modi diversi di sembrare giuste senza esserlo.

L'animazione. «C'è» o «non c'è» non si vede ispezionando il codice: un ciclo che gira e ridisegna sempre la stessa cosa supera qualunque controllo sull'esistenza della funzione. Il banco guarda i **pixel**: a 130 ms la tela deve essere quasi vuota — polvere — e a 2,3 s deve esserci una molecola. Misurato: **532 pixel accesi → 2 334**. Se i due numeri fossero uguali l'animazione non ci sarebbe, comunque sia scritto il programma. Verifica anche che il ciclo **si fermi** quando lo si ferma: una tela rimossa con il ciclo ancora acceso è una perdita che si paga in batteria.

Gli angoli. Un numero con un «°» accanto sembra una misura. Si verificano su geometrie costruite a mano, il cui valore è noto **per costruzione** e non preso da una tabella: tetraedro regolare **109,4712°**, acqua 104,47°, CO_2 180°, BF_3 120°, ammoniaca 106,13°. Tolleranza 0,05°.

Il rifiuto. Su una struttura **piatta** (z = 0) il pulsante degli angoli deve essere spento e la ragione scritta: su coordinate 2D un angolo di legame non è un angolo di legame, e mostrarlo sarebbe l'errore peggiore dei due perché somiglia a un dato. Provato nei due versi: una struttura con z ≠ 0 non deve essere scambiata per piatta, altrimenti nessun angolo si vedrebbe mai.

La prima stesura di questo banco leggeva la barra dei comandi **dopo** aver fermato il ciclo, che la rimuove: trovava zero pulsanti e poi dichiarava «ognuno ha un nome accessibile» su un insieme vuoto. Un controllo che passa perché non ha niente da guardare è esattamente il difetto che questo progetto insegue; ora la verifica pretende due pulsanti **e** due nomi.

La selezione. L'angolo lo sceglie chi guarda, cliccando sugli atomi: uno mostra tutti gli angoli che insistono su di lui, due la lunghezza del legame, tre l'angolo con il vertice nel secondo cliccato, quattro il **diedro**. Il banco verifica tutti e quattro i casi sull'etanolo — C–C 1,509 Å, C–C–O 111,9°, H–C–C–O 60,7° — e pretende che due atomi **non legati** siano dichiarati tali invece di essere spacciati per un legame: una distanza non è un legame.

I diedri. Verificati su quattro conformazioni costruite a mano: eclissata 0°, gauche 60°, ortogonale 90°, anti 180°. Anche qui la tavola scritta a mano ha corretto un mio errore, non del programma: con le coordinate che avevo scelto il diedro vale 120° e non 60°, perché guardando lungo il legame centrale il vettore proiettato sta a (−0,5; 0,866).

L'ordine d'arrivo. «Prima lo scheletro, poi gli idrogeni» è una proprietà **misurabile**, non un'intenzione: il banco legge l'avanzamento di ogni atomo a metà formazione e pretende che ogni atomo pesante sia davanti a ogni idrogeno. Questo controllo ha trovato un difetto reale: la funzione elastica che fa il rimbalzo non è monotòna, e usandola anche come avanzamento un atomo partito prima poteva risultare «meno arrivato» di uno partito dopo. Ora le due grandezze sono separate — l'avanzamento è monotòno, il fattore elastico fa solo il rimbalzo — e il rimbalzo, che prima veniva tagliato via, si vede.

Metà dei controlli sono le *prove contrarie*, perché una pagina che non costruisce niente è fulminea: la tela WebGL deve comparire comunque, le 296 figure devono esistere tutte poco dopo, nessuna cornice deve restare vuota, la stampa non deve uscire muta e la ricerca deve filtrare anche le carte la cui figura non è ancora nata. E il banco conta le sezioni che ha attraversato: su dieci invece di 89 i tempi sarebbero ottimi e la misura priva di valore.

3.5 Sicurezza e accessibilità

Banco	Verifica
`verifica-sicurezza`	Chiavi API nei file tracciati, password in chiaro, segreti nel `wrangler.toml`, telemetria, script da domini esterni
`audit_storia`	Che nessuna credenziale sia MAI entrata nel repository: **1 551 versioni distinte di file di testo** su **390 commit**, con 9 schemi. Si rifiuta di passare su un clone superficiale
`verifica-accessibilita`	Contrasto WCAG AA, nomi accessibili, etichette dei campi, testo alternativo, gerarchia dei titoli, attributo `lang` — su 13 pagine
`audit_mobile`	Che a **390 px** la pagina non scorra in orizzontale, su tutte le 95 sezioni — **e** che nessuna sezione ritagli contenuto rendendolo irraggiungibile: un elemento più largo della sua sezione, senza un antenato che scorra, è testo presente e non leggibile. Una tabella larga dentro un contenitore scorrevole non viene segnalata
`audit_rete`	Che **nessun dato dell'utente lasci il dispositivo**: un valore spia seminato in 71 depositi, l'app usata su 6 pagine e 95 sezioni, URL, intestazioni e corpo di ogni richiesta ispezionati
`audit_copertura`	Quanti byte di JavaScript vengono **davvero eseguiti** percorrendo l'applicazione: 49,89 %, registrato e difeso

`audit_storia` — 6 controlli. `verifica-sicurezza` esamina i file *tracciati*, cioè lo stato attuale: risponde a «oggi nel repository non c'è nessuna chiave» e non alla domanda che conta su un repository pubblico. Una chiave messa in un commit e togliata nel successivo passa `verifica-sicurezza` per sempre e resta leggibile a chiunque cloni, perché git non dimentica. Se succede, il rimedio non è un commit di correzione: è revocare la chiave.

Questo banco esamina ogni *versione* di ogni file di testo mai entrata nel repository. Esito: **zero credenziali in 1 551 versioni su 390 commit**.

Due difese contro sé stesso, perché qui un numero perfetto può nascere da meno superficie o da uno schema cieco:

- **Il clone non deve essere superficiale.** Un `--depth 1` contiene una

frazione della storia: il banco girerebbe, non troverebbe niente e direbbe «nessuna credenziale in tutta la storia» avendone vista un decimo. Se trova `.git/shallow`, fallisce e dice quale comando eseguire.

- **Gli schemi si provano prima di fidarsi.** Uno schema sbagliato non trova

niente ed è indistinguibile da un repository pulito: ognuno dei 9 deve riconoscere un esempio costruito e deve *rifiutare* un quasi-esempio, ed entrambi vengono provati dentro una coda di rumore con cifre, come sarebbe in un file vero.

La seconda difesa è nata da un errore in questo stesso banco. Lo schema AWS chiedeva una cifra con `(?=.*\d)`, condizione sempre soddisfatta in un file grande perché la cifra sta più avanti: trovava 53 corrispondenze, tutte la stessa stringa `AKIAAAAAAAAAAAAAAAA` — sedici «A», la zona di zeri di un'immagine in base64. Il quasi-esempio non se ne accorgeva perché era troppo corto per contenere cifre. Ora l'identificativo AWS si giudica dalla *varietà* dei suoi sedici caratteri, e i quasi-esempi portano la loro coda.

Provato anche nell'altro verso: con un commit locale che contiene una chiave AWS di forma valida il banco fallisce nominando il blob; rimosso il commit, torna a passare.

> **L'ostacolo era dello strumento, non del problema.** Per tre versioni la copertura di codice è stata dichiarata non misurabile, con questa motivazione: «strumentare richiederebbe introdurre una build, che l'applicazione non ha». Era vero per `c8` e `istanbul`, che riscrivono il sorgente prima di eseguirlo. Ma **Chromium la raccoglie da solo**, dentro il motore, senza toccare un byte del file servito: `Profiler.startPreciseCoverage`, che Playwright espone come `page.coverage.startJSCoverage()`. Non serviva una build: serviva accorgersi che il vincolo apparteneva agli strumenti scelti.
>
> **Il primo risultato è stato 100 % su ogni file**, compresi 3,6 MB di `index.html`. Era il banco che non misurava niente, nella forma più insidiosa: un numero perfetto. V8 emette per ogni funzione un intervallo esterno con il conteggio degli ingressi, e dentro quello gli intervalli a `count: 0` per ciò che non è stato eseguito. Sommando gli intervalli positivi si somma l'intero file. Si fa il contrario: si parte dal totale e si sottrae l'unione dei vuoti.
>
> Il numero vero è **49,89 %**, e regge lo stesso patto del debito di accessibilità: registrato, e il banco fallisce se scende. Una soglia assoluta («80 %») sarebbe una cifra inventata; «non deve peggiorare» è una promessa mantenibile.

> **Perché una spia e non un elenco di richieste.** SEC-02 — «nessun dato personale lascia il dispositivo» — era verificato controllando i due *meccanismi* di uscita: variabile di telemetria vuota, nessuno script da un dominio esterno. È un'evidenza vera ma indiretta: dice «non vedo come potrebbe uscire», che non è «non è uscito». Guardare invece l'elenco degli ospiti contattati sarebbe altrettanto debole, perché conclude per assenza di prove.
>
> La spia rovescia l'onere. Si scrive una stringa irripetibile dentro i dati dell'utente — note, chat, chiavi API, impostazioni, progressi — poi si **usa** l'applicazione, e ogni richiesta che esce viene aperta e letta. Se la stringa non compare da nessuna parte, è perché davvero non è uscita.
>
> ```
> BSI_PROVA_FUGA=1 node tools/banchi/audit_rete.js   # deve FALLIRE
> ```
>
> Il banco porta con sé la prova di funzionare: con quella variabile provoca una fuga di proposito, verso un ospite **dichiarato** — contattarlo è lecito, mandargli i dati dell'utente no. Se in quel caso non fallisce, è lo strumento a essere guasto.

> **Perché un banco sul traboccamento orizzontale.** UI-03 dice «l'app deve funzionare a 390 px», e la matrice lo dava per verificato da `audit_stabilita`, che però in viewport telefono guarda gli **errori JavaScript**: una pagina può non averne uno solo e uscire lo stesso di centosessantatré pixel dallo schermo. Era un requisito dichiarato coperto e non misurato. Il banco misura `scrollWidth` del documento — esattamente ciò che fa comparire la barra — e non un elemento più largo dello schermo, che dentro un contenitore fatto per scorrere è corretto e riempirebbe l'uscita di rumore finché nessuno la legge più.

> **Cosa non copre la verifica di accessibilità**, e va detto: il testo dentro gli SVG (il colore viene da `fill`, lo sfondo è una forma disegnata) e il testo su sfondi a gradiente (non hanno *un* colore). Gli elementi saltati vengono contati e riportati, non nascosti.

> **Una sezione per volta.** L'ispezione salta gli elementi non visibili — ed è corretto: un elemento nascosto non ha contrasto da misurare. Ma `index.html` alterna 95 sezioni e ne mostra una sola: su **19 751** elementi di testo il banco ne guardava **41**, e stampava «0 difetti». Non era un risultato falso, era un risultato su un campione che nessuno aveva dichiarato. Ora le sezioni vengono aperte una per una, e **il numero di sezioni percorse viene stampato**.
>
> Quel conteggio si è guadagnato lo stipendio alla prima esecuzione: il primo tentativo ne percorreva **zero** — il clic di Playwright aspetta che l'elemento sia visibile, e i pulsanti stanno dentro gruppi di navigazione richiusi — e il banco l'ha detto («95 sezioni presenti, nessuna percorsa») invece di stampare un altro zero rassicurante.

3.5-bis Il debito di accessibilità, e il patto che non cresca

Percorrendo tutte le sezioni sono emersi **1 069** difetti di contrasto. Oggi sono **zero** — l'elenco delle cause e dei rimedi è in `docs/09` §4 — e i campi privi di etichetta sono scesi da 40 a **zero**.

Il meccanismo descritto qui sotto resta, e conta più del numero che custodisce. Pretendere zero *da subito* avrebbe lasciato due sole vie, entrambe cattive: la verifica rossa per sempre, oppure allentata finché torna verde. La terza via è **dichiarare il numero** e impedirgli di crescere, qualunque esso sia. È così che da 1 069 si è arrivati a zero: ogni scalino registrato, nessuno annullabile di nascosto.

File	Ruolo
`docs/evidence/accessibilita-riferimento.json`	Il conteggio registrato

Situazione	Esito del banco
Il conteggio **sale**	✗ FALLITO — è una regressione
Il conteggio **scende**	✓ passa, e chiede di aggiornare il riferimento
Il conteggio è **uguale**	✓ passa in silenzio: debito dichiarato, non cresciuto — oggi quel valore è **0**, e il banco lo difende

```
node tools/verifica-accessibilita.js --aggiorna-riferimento
```

Da usare **solo dopo aver ridotto** i difetti, mai per far tornare verde una regressione. Anche a zero non è conformità WCAG piena: restano fuori il testo dentro gli SVG e quello su fondo a gradiente, che l'automatismo non sa giudicare e che il banco **conta e riporta** a ogni esecuzione (`docs/09` D-09). È la misura onesta di ciò che si può misurare, con la garanzia che non peggiori di nascosto.

3.6 Coerenza

Banco	Verifica
`verifica_guida`	Che le promesse della documentazione esistano nel codice; assenza di marcatori di conflitto su 58 file
`verifica-documenti`	Collegamenti interni, allineamento delle versioni, coerenza dell'indice, motivazione delle deviazioni, **coerenza interna della matrice di tracciabilità** (identificativi non duplicati, totali di copertura pari agli identificativi realmente definiti)
`genera-pacchetti`	Riscrive i riferimenti dei documenti estratti e **verifica che nessun collegamento resti rotto** nei tre pacchetti di consegna. Verifica anche che **nessun PDF sia più vecchio del documento da cui viene** e che il rapporto di verifica imbustato dichiari **questa** versione: un impaginato o un'evidenza della versione precedente, in un pacchetto di adesso, è una bugia silenziosa
`genera-pdf`	Converte i documenti in PDF e fallisce se un file risulta sotto la soglia di plausibilità (conversione a vuoto)
`verifica-affermazioni`	Confronta ogni numero dichiarato nei documenti — sezioni, farmaci, malattie, tumori, strategie, moduli — con quello **misurato nell'applicazione in esecuzione**, in italiano *e* in inglese
`test_stack`	Che lo stack React/FastAPI esegua **gli stessi file** dell'applicazione: confronta le impronte SHA-256 che `GET /salute` dichiara con quelle dei file su disco, verifica che i numeri coincidano con quelli del browser, e che **nessun file `.ts` / `.tsx` contenga una tabella di spostamenti** — perché è lì che nascerebbe il secondo motore

> **Perché i numeri vanno misurati, non ricordati.** Il dossier tecnico si apre con delle cifre: «63 malattie (23 tumori)», «46 strategie», «25 moduli». Sono la prima cosa che legge chi valuta, e bastano perché tutto il resto venga creduto o messo in dubbio. Nessuno le controllava: scritte una volta, restavano, mentre l'applicazione cambiava. E infatti `chimorga.html` dichiarava **«25 moduli»** in cima e **«17 moduli»** più sotto — due affermazioni sullo stesso oggetto, nello stesso file, una delle due falsa.
>
> **Una lezione pagata scrivendo il banco.** Avevo già concluso che «63 malattie» fosse un'esagerazione: nel sorgente avevo trovato un array `DISEASES` con 12 voci, e stavo per «correggere» il documento. Misurando nel DOM ho visto che il menu ne elenca davvero 63 — l'array che avevo trovato era un altro, più piccolo. **Un conteggio sul sorgente non è una misura: è un indizio.** Per questo il banco apre la pagina, apre la sezione e conta.
>
> Controlla anche che i documenti concordino **fra loro**: due documenti che dicono cose diverse sullo stesso oggetto sono un difetto anche quando uno dei due ha ragione. La traduzione inglese conta come uno di quei documenti — `docs/en/05` è rimasto a «84 sections» e fermo alla versione `bsi-v146` per tre rilasci, e nessun banco se ne accorgeva perché nessuno guardava l'insieme inglese.

> **Perché un controllo sulla matrice.** La matrice dichiarava `SCI-10` due volte — una come requisito verificato al 100 %, una come requisito esplicitamente **non** verificato — e la tabella riassuntiva sommava 36 requisiti mentre nel documento ne erano definiti 37. Entrambi gli errori stavano nel numero che un valutatore legge per primo. Ora il banco conta gli identificativi realmente presenti e fallisce se i totali scritti non corrispondono: la prova che il controllo misura davvero è che, reintrodotti i due errori uno per volta, li segnala entrambi.

---

4. Casi di prova notevoli

Alcuni banchi meritano una menzione perché verificano proprietà che un test funzionale non osserva.

Caso	Metodo	Risultato misurato
Perdite di memoria	95 sezioni × 5 giri, nodi DOM contati a ogni giro	+26 876 al primo giro (costruzione), **+0** nei quattro successivi
Memoria esaurita	`setItem` sostituito con una funzione che lancia sempre	10 pagine su 10 restano operative
Rete degradata ≠ assente	Richieste sospese 20 s, intercettazione a livello di contesto	Risposta dalla cache in **3 507 ms** (soglia 3 500)
Riproducibilità degli spettri	Doppio disegno, confronto byte a byte	Identici

Caso	Metodo	Risultato misurato
Contesti WebGL	Costruzioni del visore su 10 molecole consecutive	Da **7 a 1**
Cronologia corrotta	Due `Invio` a 500 ms di distanza	`user,assistant` anziché `user,user,assistant,assistant`
Contrasto del testo	Formula WCAG su ogni elemento con testo proprio, 13 pagine **e 95 sezioni**	19 751 elementi esaminati (erano 41): **1 069** difetti emersi, **1 069 corretti**, **0 registrati** come valore che non può crescere
Annullamento immediato	Stop premuto a 1,5 s, stato campionato ogni secondo	pulsante Invia disponibile dal **1º** secondo (era il 10º)

5. Come aggiungere un banco

Struttura minima

```
/* Un commento che spiega COSA dimostra questo banco e perché serve. */
const { chromium } = require('playwright-core');
const CHROME = '/opt/pw-browsers/chromium-1194/chrome-linux/chrome';

(async () => {
  const b = await chromium.launch({ executablePath: CHROME, args: ['--no-sandbox'] });
  // serviceWorkers:'block' se il banco modifica i file: senza, il SW
  // potrebbe servire una copia in cache e si misurerebbe il file vecchio
  const ctx = await b.newContext({ serviceWorkers: 'block' });
  const pg = await ctx.newPage();
  const err = [];
  pg.on('pageerror', e => err.push(e.message));

  let ok = 0, ko = 0;
  const att = (d, atteso, avuto) => {
    if (String(atteso) === String(avuto)) { ok++; console.log('  ✓ ' + d + '  → ' + avuto); }
    else { ko++; console.log('  ✗ ' + d + '\n      atteso: ' + atteso + '\n      avuto:  ' +
avuto); }
  };

  await pg.goto('http://127.0.0.1:8899/index.html', { waitUntil: 'load', timeout: 60000 });
  await pg.waitForTimeout(2500);

  // … misure …

  att('nessun errore JS', 0, err.length);
  await b.close();
  console.log('\n' + (ko ? '✗ ' + ko + ' FALLITI, ' : '') + ok + ' passati');
  process.exit(ko ? 1 : 0);   // il codice di uscita è ciò che conta
})();
```

Poi si aggiunge il nome alla famiglia opportuna in `tools/genera-evidenza.js`.

La regola che conta

> **Un banco che non misura nulla passa.** È il modo più insidioso di ottenere una falsa garanzia: nessun allarme, nessun fallimento, e nessuna verifica.

Perciò **ogni banco che simula una condizione deve contare quante volte la simulazione è realmente scattata, e fallire se il conteggio è zero.**

Casi realmente incontrati durante lo sviluppo della batteria:

Trappola	Come si manifestava
`page.route` non intercetta le richieste del Service Worker (serve `context.route`)	La prova «rete degradata» misurava una rete sana e concludeva in 16 ms invece di 3 500
Sondare `localStorage` esaurito con una scrittura di un byte	Dopo la saturazione un byte trova sempre posto: il banco «passava»
Cercare `[data-tab]` dove la navigazione usa `[data-s]`	Zero schede trovate, quindi zero misure — e zero fallimenti
Le tacche di un asse su `<canvas>` non sono testo	Vanno raccolte intercettando `fillText`; e la riga più in basso è il *titolo* dell'asse, non le tacche
Sonda installata su una libreria caricata **dopo**	Viene sostituita insieme alla libreria: zero chiamate contate mentre il codice funzionava
Una `var` dentro una IIFE non è `window.qualcosa`	Assegnarla da fuori crea una variabile diversa

---

6. Lacune di copertura

Dichiarate. La copertura riportata in `08-Traceability-Matrix.md` misura **quanti requisiti hanno un banco**, non quante righe di codice vengono eseguite: sono due cose diverse e non vanno confuse.

Lacuna	Situazione attuale	Raccomandazione
Copertura di codice	Misurata: **49,89 %** di istruzioni sul percorso più ampio. Resta fuori la copertura dell'intera batteria e quella di **rami**: un `if` entrato da un solo lato conta come coperto	Estendere la raccolta a ogni banco, e passare dalla copertura di istruzioni a quella di rami
Accessibilità	Automatizzata su 13 pagine e tutte le 95 sezioni: contrasto WCAG, nomi accessibili, etichette, testo alternativo, gerarchia dei titoli. Restano fuori il testo negli SVG e quello su gradiente, **contati** a ogni esecuzione	Affiancare `axe-core` per le regole che questo banco non implementa (ruoli ARIA, ordine di tabulazione, gestione del fuoco)
Sicurezza	`verifica-sicurezza` esegue 10 controlli su 368 file tracciati e copre SEC-01, SEC-03, SEC-05, SEC-06; SEC-04 è coperto da `verifica_guida`. **SEC-02 resta indiretto**: vedi `docs/09` D-03	Osservare il traffico di rete durante un uso reale, l'unica verifica diretta di SEC-02

Lacuna	Situazione attuale	Raccomandazione
Browser diversi da Chromium	Nessuna prova automatica su Firefox o WebKit	Estendere i banchi principali a `webkit`, dove le differenze su IndexedDB e Service Worker sono maggiori
Prestazioni	Prove manuali cross-device	Misura automatica del tempo di primo disegno
Regressione visiva	Assente	Confronto di schermate per i grafici, che sono il cuore del prodotto

---

7. Criteri di accettazione

Una versione non viene pubblicata se uno solo di questi non è soddisfatto.

- ☐ `node tools/genera-evidenza.js` si chiude con **0 falliti**
- ☐ `tools/verifica-farmaci.js`: nessun difetto, ogni deviazione registrata con il motivo
- ☐ `verifica_guida`: nessuna promessa della documentazione priva di riscontro
- ☐ Nessun errore JavaScript non di rete su tutte le pagine
- ☐ Rapporto di evidenza rigenerato sul commit che si pubblica

---

Documento aggiornato alla versione `bsi-v201`.